

LAB 05 · CAPSTONE I · DEMO DAY

Deploy Ghost as a Shared Operator

Take one genuinely ugly real-world process and make Ghost run it end to end — with the right gates, honest error handling, and a runbook a teammate could actually follow.

DIFFICULTY: HARD

TIME: ~90-120 MIN

REINFORCES: CAPSTONE · INTEGRATES LABS 1-4

YOU LEAVE WITH: A DEPLOYED WORKFLOW + A HANDOFF RUNBOOK

OBJECTIVES

- Turn a messy, real end-to-end process (onboarding, invoicing, support triage, or weekly reporting) into a numbered workflow Ghost can operate.
- Run that workflow on a real, messy case — and then deliberately break it to find where it fails.
- Make Ghost handle broken input *safely*: pause, ask, or flag instead of guessing or fabricating.
- Crystallize the whole thing into a saved workflow with explicit permission gates plus a written runbook.
- Walk away with a Demo Day story: what broke, how you debugged it, how you guarded it.

Why this lab exists

Everything so far has been practice on isolated muscles. This is the capstone that fuses them. In Labs 1-4 you gave Ghost persistent memory, had it deploy real artifacts, saved reusable workflows, and set guardrails. Here you assemble all four into **one reliable system that operates a real process without you babysitting every step**.

This is deliberately the shape of the Forward Deployment Engineer job in miniature: deploy AI into a messy real workflow, debug the agent's behavior when reality doesn't match the happy path, and translate a human's needs into a *safe* automation. It is the strongest single signal you can show an employer, and for a solo builder it reads as "I can run lightweight operations without a team." That is exactly why it is the Demo Day centerpiece.

The trap to avoid: a demo that only works on the one clean input you rehearsed. A shared operator earns trust by being **predictable when things go wrong**, not by being impressive when things go right. Most of your grade — and most of the learning — lives in the "break & steer" step.

Prerequisites

- Labs 0-4 completed. You should already be comfortable with Ghost's memory, artifact deployment, saved workflows, and permission gates.
- Ghost is live and responding in your Discord workspace.
- A real process you actually understand — ideally one that annoys you weekly. Pick from: client onboarding, invoicing, support triage, or weekly reporting. Vague or invented processes make a weak capstone; use something with real edge cases.

- At least two sample cases ready: one realistic-messy "good" case, and one deliberately broken case (missing fields, a duplicate, or a weird edge).

The build — step by step

The flow is **Map** → **Run** → **Break & steer** → **Crystallize**. Do not skip ahead to saving the workflow; the reliability comes from the breaking.

1

MAP — make Ghost interview you before it automates anything

A good operator understands the process before it touches it. Rather than dictating steps, ask Ghost to interview you. This surfaces the tacit knowledge you would otherwise forget to encode.

```
# IN DISCORD, TO GHOST
```

```
Before you automate anything, interview me about this process so you  
understand it fully. I want to hand you my client onboarding.
```

```
Ask me, one topic at a time:
```

- ```
- what TRIGGERS this process (what event starts it)
- every step, in order, and who/what does each today
- the inputs each step needs and the outputs it produces
- where it goes WRONG or gets stuck today
- and the big one: what must NEVER happen automatically,
 without a human approving first
```

```
Don't propose a solution yet. Just ask until you're confident.
```

Answer honestly, including the ugly parts. When Ghost has enough, have it write the process back to you:

```
AFTER THE INTERVIEW
```

```
Now write my process back to me as a NUMBERED workflow.
Mark every decision point and every place a human must approve
before you continue. Save nothing yet — I want to review it first.
```

Read the numbered version critically. If Ghost misunderstood a step or missed a decision point, correct it now. The map is the contract for everything that follows.

## RUN — execute the workflow on a real, messy case

Do not test on a sanitized example. Feed Ghost a genuine case with the normal amount of real-world mess — an oddly formatted name, a note in the wrong field, an attachment referenced but not clean.

```
RUN THE MAPPED WORKFLOW LIVE
```

```
Run the workflow on this real case. Narrate what you're doing at
each numbered step, and STOP at every approval point and wait for me.
```

```
New client: "acme co." — contact reply below, pasted as-is:
```

```
"hey we're ready to start, budget approved, can u send the kickoff doc?
oh and invoice goes to finance not me. -J (from Acme, the packaging one)"
```

Watch two things: does Ghost follow the numbered steps in order, and does it actually pause where the map said it should? If it deploys a real artifact (a kickoff doc, a summary page), confirm the artifact is correct and that it halted for your approval before anything left the workspace.

## BREAK & STEER — the reliability lesson

This is the heart of the lab. Feed Ghost a **broken** case and watch how it behaves under stress. The goal is not a clean run — the goal is to catch failure modes and fold each one back into the workflow as a rule.

```
DELIBERATELY BROKEN INPUT
```

Now run this case:

```
"Client: (blank). Budget: ??? Contact: same as the Acme one from
earlier, I think. Start date: yesterday. Also I already sent them
a kickoff doc last week, resend it?"
```

```
Fields are missing, there's a possible duplicate, and a contradiction.
Do NOT guess or fill anything in silently.
```

The pass/fail bar: Ghost must **ask or flag** rather than fabricate the missing client name, must **catch the possible duplicate** instead of blindly creating a second record, and must **not crash or silently drop the case**. If it invents a plausible-looking value, that is the exact failure this lab is designed to expose.

Every failure mode you find becomes a rule. Steer Ghost to update the map:

```
FOLD THE FAILURE BACK IN
```

```
Good — you flagged the missing client name instead of guessing.
```

```
Add these as permanent rules in the workflow:
```

1. If a required field is empty, PAUSE and ask; never infer it.
  2. Before creating a client, check memory for a matching record;  
if one might exist, ask me to confirm new-vs-duplicate.
  3. If two inputs contradict each other, surface both and ask which wins.
- ```
Then re-run the broken case so I can confirm the rules hold.
```

Repeat: throw one or two more broken cases (a malformed amount, an out-of-order trigger) and harden the workflow each time. You are done when a fresh broken case gets handled the way you'd want a careful teammate to handle it.

CRYSTALLIZE — save the workflow with gates, then write the runbook

Now make it durable. Save the hardened process to Ghost's own workflows with the permission gates written in explicitly.

```
# PERSIST THE OPERATOR
```

```
Save this as a workflow called "client-onboarding-operator".
Bake in the permission gates: pause for my approval before
(a) sending anything to the client, (b) creating a new client record,
and (c) issuing or routing an invoice. Everything else you may do,
but always narrate it.
```

Then have Ghost produce the handoff document — the artifact that turns your one-off demo into something a teammate could run.

```
# THE RUNBOOK
```

```
Write a RUNBOOK a teammate could follow without me. Include:
- what this workflow does, in two sentences
- what TRIGGERS it
- every point where it PAUSES for human approval, and why
- the failure modes we found and exactly how each is handled
- what to do WHEN UNSURE (the default: stop and ask, don't guess)

Deploy it as a document I can share, and keep a copy in memory.
```

Review the runbook as if you were the teammate. If any step assumes knowledge only you have, send it back for a revision. A runbook that only you can follow is not a runbook.

THE BROKEN-CASE TEST

Your workflow is only as reliable as its worst input. Before you present, run the broken case one final time cold and confirm Ghost does **all** of the following:

- **Never fabricates.** A missing client name, budget, or date triggers a question — not an invented value that looks real.
- **Catches duplicates.** A possible repeat record is surfaced for confirmation, not silently created twice.
- **Refuses to auto-cross a gate.** Nothing is sent to a client, no invoice is routed, and no record is created without explicit approval.
- **Fails loud, not silent.** A contradiction or malformed field produces a clear flag, not a dropped case or a crash.

If Ghost guesses even once, that is not a passing operator — go back to Break & Steer and add the missing rule.

DELIVERABLE

Bring three things to Demo Day: the deployed workflow, the runbook, and the story.

- ☐ A saved Ghost workflow that operates your real process end to end, with permission gates baked in.
- ☐ A deployed, shareable runbook covering what it does, its trigger, its approval pauses, the failure modes found, and the "when unsure" default.
- ☐ A successful live run on a real messy case, halting correctly at every gate.
- ☐ A clean handling of at least one broken case (missing field, duplicate, or contradiction) with no fabrication.
- ☐ Your Demo Day narrative written down: *what broke, how I debugged it, how I guarded it.*

Stretch goals

- **Metrics line.** Have Ghost track and report how many cases it handled and how many it escalated to you, so the operator has a visible reliability record.
- **Rollback step.** Extend the runbook with a documented way to undo the last action if something is caught after the fact — every real operator needs an "oops" path.
- **Self-audit.** Ask Ghost to review its own workflow for gaps: "Where could you still act without approval? What input would still break you?" Then close whatever it finds.

FACILITATOR NOTES

This is the Part I capstone and the Demo Day anchor — budget for it and let the strong projects present.

- The most common weak submission is a workflow that only ran on the rehearsed clean case. Push every student to demo the **broken** case; that is where the reliability skill actually shows.
- Watch for silent fabrication — a student may not notice Ghost invented a client name because the output looks plausible. Have them diff Ghost's output against the actual input.
- Steer students to pick a process they truly understand. Invented processes produce shallow edge cases and a thin runbook.
- Grade the runbook as if you were the teammate inheriting it. If you couldn't run the process from the document alone, it isn't done.
- Frame the Demo Day pitch around the FDE narrative: deploying AI into a messy workflow, debugging agent behavior, and guarding it. That framing is the employer signal — and the solo-builder version is "I run operations without a team."
- Keep everything inside Ghost: interaction in Discord, artifacts and runbook deployed from Ghost's workspace, workflow and memory saved to the student's own Ghost. No local files.